第一阶段成果汇报

本科毕业设计 · 核心设计 / 代码实现 / 论文前半部分

面向代码修复的多智能体协作与验证系统设计与实现

Design and Implementation of a Multi-Agent Collaboration and Verification System for Code Repair

项目载体： yeCoding Python ｜ 汇报重点：设计可解释、代码可定位、结果可核查

研究问题与第一阶段目标

01 研究定位

任务输入

问题描述 + 当前源码
预备镜像 + 固定测试
尝试 / 调用 / Token / 时间预算

任务输出

候选补丁 + 验证证据
明确终止状态 + 消耗记录
原始仓库不自动修改

研究问题	本阶段对应交付
RQ1： 怎样约束修改与执行?	双副本工作区、受限容器、清理机制
RQ2： 怎样形成可验证的协作闭环?	角色权限、固定测试、反馈与状态记录
RQ3： 多角色带来什么收益与成本?	对比方案与初步证据，保留失败结果

适用范围：具备可运行测试的 Python 修复任务；不涉及模型训练或并行补丁合并。

依据：论文第 1 、 3 章； agent/repair.py

文献依据如何转化为设计选择

研究方向	已有启发	本课题的实现选择
工具型 Agent ReAct / SWE-agent	观察、行动、执行反馈	复用 query 与 ToolExecutor 把验收提升到流程控制层
多角色协作 AutoGen	任务分工与上下文隔离	协调 → 修改 → 审查 顺序交接，单个写入者
Automatic Program Repair 补丁过拟合与独立评估	测试通过不等于语义正确	固定验收 + 事后追加测试 区分候选正确性与流程完成

研究假设保持开放：多角色可能改善检查，也可能因交接成本消耗预算。

依据：论文第 2 章； references.bib 中 ReAct / SWE-agent / AutoGen / APR 与补丁评估文献

系统总体架构

03 完整设计 · 四个逻辑层，运行于同一 Python 应用

接口层	CLI repair / REPL /repair / SDK / Runtime + Task Worker
编排层	QueryEngine：通用 ReAct / Plan / Team | 专用 VerifiedRepair
能力层	query 模型循环 + 工具执行器 + AST 代码上下文 + Skill / Memory
执行与存储层	RepairWorkspace 双副本 + DockerSandbox + JSON / JSONL / Patch

入口差异： CLI → VerifiedRepair； REPL / SDK / Runtime → QueryEngine.repair → 同一核心。

依据： agent/query_engine.py； agent/repair.py； entrypoints/cli.py

模块组成与核心代码位置

03 完整设计 · 现有平台与本课题新增工作的边界

模块	代码路径（相对 src/yecoding/ ）	责任 / 工作性质
VerifiedRepair	agent/repair.py	专用闭环、角色交接、预算与证据；新增
RepairWorkspace	sandbox/workspace.py	源码快照、候选检查、双副本；新增
DockerSandbox	sandbox/docker.py	命令隔离、资源限制、清理；新增
入口适配	entrypoints/ ； agent/query_engine.py	CLI / REPL / SDK / Runtime 接线
模型与工具循环	agent/query.py ； llm/ ； tools/	复用，按角色收紧注册工具
代码上下文	rag/	复用 AST 索引与分层上下文
知识与经验	skill/ ； memory/ ； builtin_skills/	复用机制，新增修复 Skill 与接入约束

本课题重点：把已有编码能力组织为可约束、可验收、可追溯的专用修复流程。

依据：源码目录；论文第 3 、 4 、 5 章

修复闭环：从失败复现到补丁交付

1 输入与副本

校验选项
快照 + 指纹

2 基线复现

固定命令执行
确认真实失败

3 协调与修改

计划 → Worker
真实文件变更

4 候选检查

保护文件 + diff
重建 verify

5 测试与审查

固定测试
Verifier 交接

6 决策与交付

成功 / 重试 / 停止
报告 + 补丁

失败且预算允许 → 反馈返回 Worker；修改副本保留，角色对话重新建立。

分支：基线已通过 ≠ 修复成功；环境异常单列；空补丁 / 重复补丁停止为 no_progress 。

依据： agent/repair.py： VerifiedRepair.run

三角色协作与权限契约

03 完整设计 · 顺序交接、单写者、独立上下文

Coordinator · 协调

输入：任务、约束、经验
工具：只读与代码搜索
工作区： work
交付：简短修复计划

Worker · 修改

输入：任务、计划、反馈
工具：读写 + 容器命令
工作区： work
交付：文件变更与摘要

Verifier · 审查

输入：候选 diff 、证据
工具：只读与代码搜索
工作区： verify
交付：问题分析与建议

每次角色调用： history=[] + 独立 SkillContextBuffer

跨角色传递结构化任务、计划、候选和证据；不传递共享的可写聊天状态。

权限通过工具注册表落实；角色提示词负责指导行为，但不是安全边界。

依据： agent/repair.py： _agent / _registry

工作区与候选：原仓库不作为试验场

03 完整设计 + 源码 ①

原始源码 → 快照

路径 + 文件字节 + SHA256；只读取

work 修改副本

Worker 编辑；跨尝试保留修改

候选检查 → verify

检查保护文件；合法候选重建验证副本

独立交付目录

报告 / 事件 / 尝试补丁；成功才导出最终补丁

```
current = inventory(self.work)
changes = [
    name
    for name in sorted(self.base.keys() | current.keys())
    if self.base.get(name) != current.get(name)
]
forbidden = [name for name in changes if protected(name)]
if forbidden:
    # reject protected-file edits
    ...
```

展示节选：省略异常消息，保留原始检查逻辑。
测试与配置禁止变化；不支持二进制候选与符号链接。

验证对象始终是候选文件本身，不是 Worker 的“已修好”描述。

依据： sandbox/workspace.py： RepairWorkspace.__init__ / candidate ； 完整原始节选见口播稿

工具边界：按角色构造能力白名单


```
registry = ToolRegistry()
for tool in get_builtin_tools():
    if tool.name not in READ_TOOLS | (set() if readonly else WRITE_TOOLS):
        continue
```

边界	落实方式
角色可用工具	READ_TOOLS / WRITE_TOOLS；只读角色不注册写入与命令
文件路径与保护文件	PathGuard + protected；命令改动仍会在候选阶段检查
命令执行	bash / execute_command handler → DockerSandbox
外部工具与会话状态	专用修复配置副本；不暴露任意 MCP / web / 记忆写工具

失效即停止：容器镜像缺失或启动异常，不降级为宿主机 shell。

依据：agent/repair.py： _registry； run 中的复制配置与策略调整

容器执行边界与资源约束

03 完整设计 · 容器限制命令，宿主机保留编排与模型通信

Host · 宿主机

VerifiedRepair 编排与预算
LLM 通信、文件工具、 AST 索引
快照、候选检查、记录落盘



Docker · 每条命令一个容器

Worker： work 可写挂载
固定测试： verify 只读挂载
执行结束 / 超时 / 取消 → 清理

隔离配置	资源与生命周期
--network=none ； --read-only	--memory=512m ； --cpus=1
--cap-drop=ALL ； no-new-privileges	128 个进程； /tmp 为 64 MiB tmpfs
--pull=never ；只挂载任务副本	stdout / stderr 各保留 20,000 bytes ；持续排空

边界声明：共享内核；依赖预备镜像；容器启动失败不会回退到宿主机执行。

依据： sandbox/docker.py： arguments / execute / _remove

共享预算与明确终止状态

03 完整设计 + 源码 ②

```
async def chat(self, *args, **kwargs):
    if (
        self.calls >= self.options.max_llm_calls
        or self.usage.total_tokens >= self.options.max_tokens
    ):
        raise BudgetExhausted("repair model call/token budget exhausted")
    self.calls += 1
    async for event in self.client.chat(*args, **kwargs):
        if event.get("type") == "usage":
            self.usage = self.usage + Usage.from_mapping(event.get("usage"))
        yield event
```

产品默认预算

3 次尝试
36 次模型调用
60,000 Token 请求前阈值
900 秒流程 / 60 秒命令

成功： verified ｜ 非成功： baseline_passed / no_progress / budget_exhausted 等

所有角色与重试共享预算；响应可能越过请求前 Token 阈值，不把它写成最终硬上限。

依据： agent/repair.py： BudgetClient / run； config.py： RepairConfig

上下文、 Skill 与 Memory 如何配合

03 完整设计 · 代码证据、过程知识与项目经验分开管理

AST 代码上下文

Python 词法符号与近似关系
签名 → 关系 → 源码 → 摘要
片段长度与来源可见
不等于完整语义调用图

Skill · 过程知识

协调： repair-reproduce
修改： reproduce + minimal
审查： repair-verify
按角色注入，记录版本与哈希

Memory · 项目经验

原项目作用域只召回一次
修复经验校验 base_hash
成功且启用时才写入
经验按不可信数据处理

任务 + 代码上下文 + 角色 Skill + 合格经验 → 独立角色上下文 → 工具执行

已完成机制接入；尚未通过独立消融实验证明 Skill / Memory 带来修复质量提升。

依据： rag/context_builder.py ； skill/ ； agent/repair.py ： _agent / _recall / _remember

验收核心：测试决定门槛，审查不能覆盖失败

04 核心实现 + 源码 ③

```
verification = await self._tests(self.workspace.verify)
passed = all(result["passed"] for result in verification)
attempt_report.update(verification=verification, passed=passed)
```

中间步骤：非 single 策略执行 Verifier 审查交接，失败 / 超预算会中断成功出口。

```
if passed:
    self.report["status"] = "verified"
    (self.artifacts / "candidate.patch").write_text(patch, encoding="utf-8")
```

verified = 固定验收通过 + 所需交接完成；不是多 Agent 投票，也不是模型自报成功。

依据： agent/repair.py： run 中 verification / review / if passed

一次真实修复的交付结果

04 核心代码与可检查产物 · 归档案例，不是现场模型演示

```
diff --git a/dotenv/variables.py b/dotenv/variables.py
--- a/dotenv/variables.py
+++ b/dotenv/variables.py
@@ -64,6 +64,8 @@
     def resolve(self, env: Mapping[str, Optional[str]]) -> str:
         default = self.default if self.default is not None else ""
         result = env.get(self.name, default)
+        if result is None:
+            return ""
         return str(result)
```

none-value · single

模型调用： 5 次
累计 Token： 21,220
固定测试： 9 / 9 通过
追加后总计： 13 / 13 通过

交付： report.json / events.jsonl / attempt-1.patch / candidate.patch

本例只证明该任务的可执行交付链路；不把单例成功外推为通用修复质量。

依据： eval/evidence/repair-runs/fcc748e586de4741a38212e5f4941de7/candidate.patch

论文前半部分：已经形成的论证链

05 论文进展 · 英文 BEng 论文，本次聚焦第 1–5 章

章节	已形成内容	对应本次汇报
1 Introduction	背景、任务定义、 RQ1–RQ3 、贡献边界	第 2 页
2 Related Technologies	Agent / APR / 补丁评估与技术选择	第 3 页
3 Requirements Analysis	功能、权限、验收、失败与预算需求	第 2 、 5 、 11 页
4 System Design	架构、角色、双副本、状态、入口契约	第 4–12 页
5 Core Implementation	候选检查、容器执行、预算与固定验收	第 8–14 页

当前状态：可讲解、可定位的论文稿；本次聚焦前部，后半已有实验仅作初步验证。

依据： docs/thesis/en/01–05 ； docs/thesis/latex/build/main.pdf

第一阶段验证：能运行，不等于已证明优越

06 已有证据与边界 · 以下为 2026-10-20 归档结果

功能验证 · 160 passed

134 项原有测试
25 项修复测试 + 1 项快照回归
脚本模型 + 真实 Docker 执行
覆盖重试、保护文件、预算、取消等

真实模型 · 24 次运行

8 个留出人工缺陷 × 3 种策略
同一项目、同一模型、每条件 1 次
single / team / verified
完成数： 8/8 · 5/8 · 4/8

关键发现： 7 次 Token 预算失败；多角色开销可能抵消收益。追加检查无已观测误接收。

不能推断：多角色一定更好、追加测试覆盖全部语义、或小样本结果可泛化到真实仓库。

依据： eval/evidence/repair-verification.json ； repair-heldout-live.json ； 论文第 6 章

下一阶段：补强证据与完善论文

07 后续安排 · 提议，待与导师确认优先级

任务	下一步产物	验收方式
扩展与重复评估	真实缺陷清单、重复运行与完整失败记录	冻结任务与划分；报告全部运行
协作开销分析	角色 Token / 调用 / 交接阶段统计	控制外部预算，比较成功与成本
知识机制消融	按需设计 Skill / Memory 开关对照	同任务同条件，隔离跨任务污染
论文与交付整理	修订结果、讨论、局限及复现说明	图表—原始记录—源码可互相定位

优先级建议：先扩大评估与解释失败，再决定是否调整架构。

依据：基于当前限制提出；尚未执行

本阶段小结与希望确认的问题

08 总结

设计	四层架构 + 三角色契约 双副本 + 受限执行 + 固定验收
代码	VerifiedRepair / Workspace / Docker 统一核心、失败状态、证据交付
论文	英文第 1–5 章可对应设计与源码 初步实验如实保留成本与失败

请导师确认　课题范围 • 真实缺陷与重复实验的优先级 • 论文论证重点

备份 A： 三种策略的实际结果与设置

技术追问备用 · 不计入主讲时间

策略	流程成功	追加检查通过 *	平均累计 Token	角色 / 后续反馈
single	8/8	8/8	31,120	Worker / 执行
team	5/8	5/8	64,541	三角色 / 审查
verified	4/8	5/8	60,167	三角色 / 执行 + 审查

设置： python-dotenv v1.1.1； deepseek-v4-flash； temperature=0；每响应最多 2,048 Token 。
预算： 3 次尝试、 18 次调用、 60,000 Token 请求前阈值、 180 秒流程、 60 秒命令。
知识： Skill 开启、跨任务 Memory 关闭； 8 个留出人工缺陷，每条件仅运行一次。

* 18 个可用补丁的追加检查均通过；其余 6 次无补丁，未执行追加检查，不标成测试失败。

依据： repair-heldout-live.json： summary / records；论文第 6 章

备份 B： 状态与公开入口契约

技术追问备用 · 不计入主讲时间

状态	含义
verified	固定测试通过，所需交接完成，导出 candidate.patch
baseline_passed	基线已通过，没有建立成功修复
environment_error	执行环境异常或单命令超时
no_progress	空候选或连续相同候选
attempts_exhausted	尝试数耗尽
budget_exhausted	调用 / Token 请求前阈值触发
timeout / canceled / error	流程截止 / 取消 / 其他异常

CLI： 仅 verified → exit 0； REPL： /repair； SDK： QueryEngine.repair ，调用方检查 status 。
Runtime： mode=repair ；非成功报告映射为 error ；取消状态不被迟到结果覆盖。

依据： agent/repair.py ； entrypoints/cli.py ； runtime/api.py / tasks.py

备份 C： 关键文献与材料位置

技术追问备用 · 题名及出版信息来自论文参考文献库

[1] Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.

[2] Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 37, 2024.

[3] Wu et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. COLM, 2024.

[4] Gazzola et al. Automatic Software Repair: A Survey. IEEE TSE, 45(1): 34–67, 2019.

[5] Yu et al. Alleviating Patch Overfitting with Automatic Test Generation: A Study of Feasibility and Effectiveness for the Nopol Repair System. EMSE, 24(1): 33–67, 2019.

[6] Ye et al. Automated Patch Assessment for Program Repair at Scale. EMSE, 26(2), Article 20, 2021.

代码： src/yecoding/agent/repair.py ； sandbox/
论文： docs/thesis/en/ ； docs/thesis/latex/build/main.pdf
证据： eval/evidence/repair-verification.json ； repair-heldout-live.json

依据： docs/thesis/latex/references.bib ； docs/thesis/en/ ； eval/evidence/